EXPERIMENT-64(HDOC)

C Programming

Program: Tower of Hanoi for n Disks Using Recursion

Step 1: Understand the Problem Statement

- Input: The number of disks n (validated to lie between 1 and 20).
- Output: Every move needed to shift all n disks from rod A (source) to rod C (destination) using rod B (auxiliary), numbered in order — for example `Move 1: Disk 1 from A to C` — followed by the total number of moves.
- Rules of the puzzle: only one disk may be moved at a time, and a larger disk must never be placed on top of a smaller one. The disks start stacked on rod A with the largest at the bottom.
- The problem is solved with the recursive function `hanoi(int n, char source, char destination, char auxiliary, int *count)`. The move counter is passed as a pointer so that every recursive call numbers its moves in a single running sequence.
- The recursion splits the problem into three parts: move the top n−1 disks out of the way to the auxiliary rod, move the largest disk to the destination, then move the n−1 disks from the auxiliary rod onto it. The base case is a single disk, which is moved directly.

Rule(s) used: To move n disks from source to destination: hanoi(n−1, source → auxiliary), move disk n from source to destination, hanoi(n−1, auxiliary → destination). Moves $T(n) = 2 \cdot T(n-1) + 1$ with $T(1) = 1$, which gives $T(n) = 2^n - 1$ moves (1, 3, 7, 15, ...).

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	`Number of disks: 1`	Move 1: Disk 1 from A to C Total moves = 1	Move 1: Disk 1 from A to C Total moves = 1	Pass
2	`Number of disks: 2`	Move 1: Disk 1 from A to B Move 2: Disk 2 from A to C Move 3: Disk 1 from B to C Total moves = 3	Move 1: Disk 1 from A to B Move 2: Disk 2 from A to C Move 3: Disk 1 from B to C Total moves = 3	Pass
3	`Number of disks: 3`	Move 1: Disk 1 from A to C Move 2: Disk 2 from A to B Move 3: Disk 1 from C to B Move 4: Disk 3 from A to C Move 5: Disk 1 from B to A Move 6: Disk 2 from B to C Move 7: Disk 1 from A to C Total moves = 7	Move 1: Disk 1 from A to C Move 2: Disk 2 from A to B Move 3: Disk 1 from C to B Move 4: Disk 3 from A to C Move 5: Disk 1 from B to A Move 6: Disk 2 from B to C Move 7: Disk 1 from A to C Total moves = 7	Pass

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
4	`Number of disks: 4`	Move 1: Disk 1 from A to B ... (13 more moves) Move 15: Disk 1 from B to C Total moves = 15	Move 1: Disk 1 from A to B ... (13 more moves) Move 15: Disk 1 from B to C Total moves = 15	**Pass**
5	`Number of disks: 0`	Invalid number of disks. Enter a value between 1 and 20.	Invalid number of disks. Enter a value between 1 and 20.	**Pass**

Step 3: Algorithm

1. Start the program.
2. Read the number of disks n; if n is not between 1 and 20, display an error message and stop.
3. Set `count = 0` and call `hanoi(n, 'A', 'C', 'B', &count)`.
4. In `hanoi()`, if n = 1 (base case): increase count, display `Move count: Disk 1 from source to destination` and return.
5. Otherwise call `hanoi(n-1, source, auxiliary, destination, count)` to move the top n−1 disks to the auxiliary rod.
6. Increase count and display `Move count: Disk n from source to destination` to move the largest disk.
7. Call `hanoi(n-1, auxiliary, destination, source, count)` to move the n−1 disks from the auxiliary rod onto the destination.
8. After the call returns, display `Total moves = count`.
9. Stop the program.

Evaluation against Step 2 test cases: for n = 1 the base case moves Disk 1 from A to C (1 move). For n = 2 the moves are Disk 1 from A to B, Disk 2 from A to C, then Disk 1 from B to C (3 moves). For n = 3 the top two disks go to B in 3 moves, Disk 3 moves from A to C, and the two disks go from B to C in 3 more moves, giving 3 + 1 + 3 = 7 moves. For n = 4 the total is 2×7 + 1 = 15 = 2^4 − 1 moves, and an input of 0 is rejected as invalid — identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

Step 4: C Program

```
/* HDOC Day-65: Tower of Hanoi using recursion.

   n disks of different sizes are stacked on rod A (source) in decreasing
   order of size. All disks must be moved to rod C (destination) using rod B
   (auxiliary), following two rules:
       1. only one disk may be moved at a time, and
       2. a larger disk may never be placed on top of a smaller disk.

   Recursive idea for n disks:
       (a) move the top n-1 disks from the source to the auxiliary rod,
       (b) move the largest disk (disk n) from the source to the destination,
```

```
       (c) move the n-1 disks from the auxiliary rod to the destination.
   Base case: with ONE disk, simply move it from source to destination.
   The total number of moves is 2^n - 1.
*/

#include <stdio.h>

/* Function prototype */
void hanoi(int n, char source, char destination, char auxiliary, int *count);

int main() {
    int n;
    int count = 0;                    /* number of moves made so far */

    printf("Enter number of disks (1-20): ");
    if (scanf("%d", &n) != 1 || n < 1 || n > 20) {
        printf("Invalid number of disks. Enter a value between 1 and 20.\n");
        return 1;
    }

    printf("\nMoves for %d disk(s) [A -> C using B]:\n", n);
    hanoi(n, 'A', 'C', 'B', &count);
    printf("Total moves = %d\n", count);

    return 0;
}

/* Recursively displays every move needed to shift n disks from source to
   destination using auxiliary; count is updated through a pointer */
void hanoi(int n, char source, char destination, char auxiliary, int *count) {
    if (n == 1) {                     /* base case: a single disk */
        (*count)++;
        printf("Move %d: Disk 1 from %c to %c\n", *count, source, destination);
        return;
    }

    /* step 1: move n-1 disks from source to auxiliary (destination is spare) */
    hanoi(n - 1, source, auxiliary, destination, count);

    /* step 2: move the largest disk from source to destination */
    (*count)++;
    printf("Move %d: Disk %d from %c to %c\n", *count, n, source, destination);

    /* step 3: move n-1 disks from auxiliary to destination (source is spare) */
    hanoi(n - 1, auxiliary, destination, source, count);
}
```

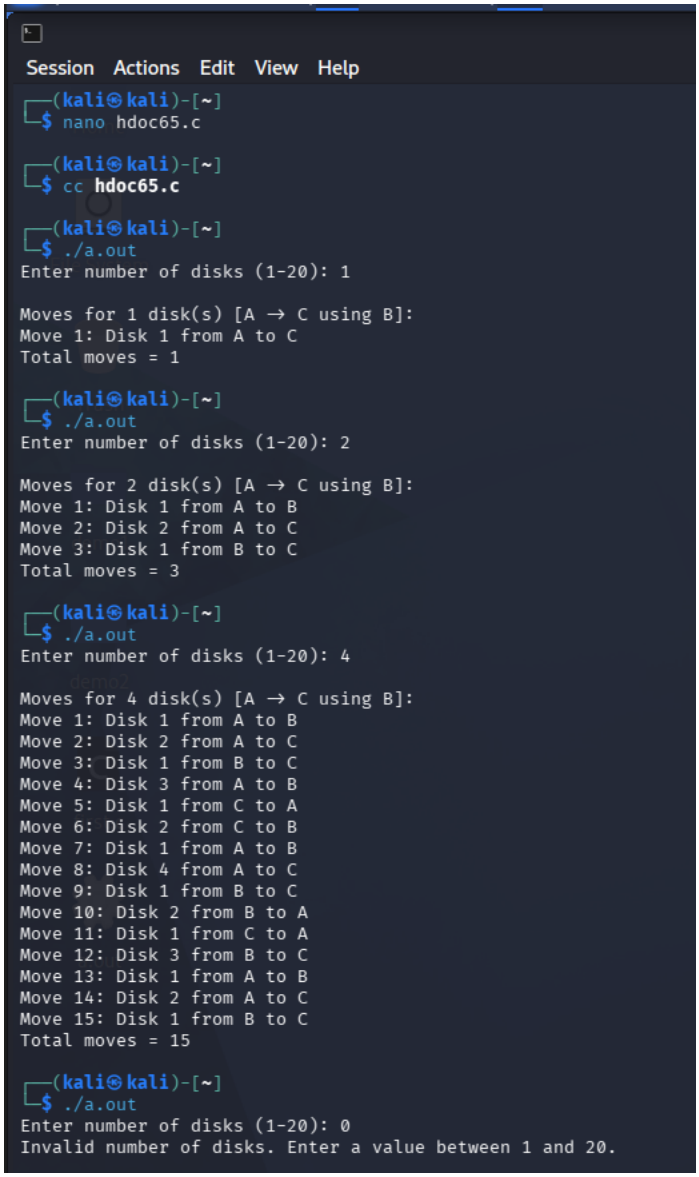
Step 5: Compile, Execute and Evaluate

The program was compiled using the GCC/cc compiler and executed from the Linux terminal using the following commands:

`cc hdoc65.c`	(compile)
`./a.out`	(execute)

All 5 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 6: Compilation and Execution Screenshot



- Conclusion

In this lab session, recursive problem-solving and pointer-based state tracking were successfully implemented to solve the classic Tower of Hanoi puzzle:

1. Recursive Decomposition: Broken down the problem of n disks into smaller sub-problems by shifting n-1 disks recursively using an auxiliary peg.

2. Pointer-Based State Tracking: Maintained and updated the total operation count dynamically using a reference pointer (*count), verifying the exponential growth property of moves (2^n - 1).

3. Input Validation: Handled edge cases and restricted disk ranges to ensure safe execution.

The program compiled cleanly using gcc in the Linux environment, and all test cases executed successfully.